

Message-Oriented Middleware: MQTT e AMQP

Tecnologias de Comunicação para a Internet das Coisas:
Sebenta da Aula 12

Catarina Silva

2026

Abstract

Este documento acompanha a décima segunda aula da unidade curricular e trata os middlewares orientados a mensagens: o paradigma de desacoplamento entre produtores e consumidores, o protocolo AMQP e o protocolo MQTT, com comparação entre ambos e critérios de seleção. Destina-se ao estudo autónomo e ao apoio ao guião prático sobre RabbitMQ.

Table of Contents

1 O Paradigma Orientado a Mensagens

Este capítulo explica que problema os middlewares orientados a mensagens resolvem e apresenta os dois modelos de distribuição que suportam.

1.1 Motivação e Conceito

A escala de uma solução impõe flexibilidade na sua arquitetura: o que funciona com dez sensores raramente funciona com dez mil. Os modelos de programação convencionais, em que os componentes se invocam diretamente uns aos outros, escalam muito mal, cada novo consumidor de dados obriga a

alterar o produtor, e cada produtor tem de conhecer todos os destinos possíveis da informação que gera. A computação em nuvem oferece capacidade de escalamento dinâmico notável, mas as aplicações têm de estar construídas de forma a poder tirar partido desse modelo.

O modelo de mensagens resolve o problema pelo desacoplamento dos componentes, e é esse desacoplamento que torna possível aumentar a escala. Não é por acaso que a maioria das infraestruturas de nuvem já integra middlewares orientados a mensagens, como o RabbitMQ. Concretamente: quem produz os dados deixa de precisar de saber quem os consome, quantos consumidores existem, ou sequer se estes estão disponíveis no momento da publicação.

Um Message-Oriented Middleware é uma camada de software que se interpõe entre produtores e consumidores de dados, assumindo a responsabilidade pelo encaminhamento das mensagens. O elemento central é o *broker*, o servidor que recebe as mensagens dos produtores e as entrega aos consumidores adequados.

O desacoplamento que este modelo proporciona opera em três dimensões. No espaço, produtor e consumidor não precisam de conhecer os endereços um do outro. No tempo, não precisam de estar ativos em simultâneo, uma vez que o broker retém as mensagens. Na sincronização, o produtor não fica bloqueado à espera que o consumidor processe a mensagem. Numa solução de IoT, este é precisamente o ponto em que os dados recolhidos por um gateway deixam de estar confinados ao processo que os recolheu, o passo que foi antecipado no trabalho prático sobre notificações BLE.

1.2 Modelos de Distribuição

Existem dois modelos fundamentais de distribuição de mensagens, com finalidades distintas.

No modelo de filas, cada mensagem é colocada numa fila e entregue a um único consumidor. Vários consumidores podem retirar da mesma fila, repartindo entre si a carga de trabalho. Este modelo é adequado à distribuição de tarefas, em que cada unidade de trabalho deve ser processada exatamente uma vez, se dois consumidores processassem a mesma tarefa, o resultado seria duplicação indevida.

No modelo de publicação-subscrição, cada mensagem publicada num tópico é entregue a todos os consumidores subscritos desse tópico. Este modelo é adequado à difusão de dados de sensorização, em que a mesma leitura pode interessar simultaneamente a um mecanismo de alarmística, a um módulo de análise e a um sistema de armazenamento.

O AMQP suporta ambos os modelos; o MQTT assenta exclusivamente em publicação-subscrição. Em soluções de IoT, o segundo modelo é o mais frequente, precisamente por permitir acrescentar consumidores sem alterar os produtores, capacidade que corresponde ao requisito de escalabilidade enunciado no enunciado do mini-projeto.

2 AMQP

O AMQP é a norma aberta de referência para middlewares de mensagens em contextos empresariais. Este capítulo trata o seu modelo, garantias e implementações.

2.1 Modelo e Funcionamento

No AMQP, todos os recursos são criados e destruídos dinamicamente pelos clientes, à medida das necessidades: não existe pré-configuração estática do broker. O modelo conceptual é simples e limpo, assentando em apenas três conceitos fundamentais. Trata-se de uma norma aberta, desenvolvida pelo grupo de trabalho AMQP, com bibliotecas de cliente disponíveis gratuitamente

para muitas linguagens de programação, e com o RabbitMQ como implementação de referência de código aberto, o broker que será utilizado no guião prático desta aula.

Os três conceitos centrais são a mensagem, o tipo de troca (*exchange*) e a fila. O fluxo de uma mensagem percorre-os pela seguinte ordem. O produtor publica uma mensagem, que é entregue a uma *exchange* no broker. É esta que decide, com base no seu tipo e nas regras de ligação configuradas, para que filas a mensagem deve seguir. A mensagem é depois colocada na fila ou filas correspondentes, onde aguarda até ser consumida. O protocolo garante consumo ordenado das mensagens dentro de cada fila.

O modelo permite que múltiplos consumidores recebam mensagens de um mesmo tópico, assegurando débitos de consumo distintos entre eles, um consumidor lento não trava os restantes. As mensagens são encaminhadas entre diferentes consumidores, sendo descartados os tópicos que não tenham subscritores. Os tópicos admitem caracteres universais, o que aumenta a flexibilidade do endereçamento e permite subscrever famílias inteiras de tópicos com uma única declaração.

2.2 Garantias, Abertura e Implementações

O AMQP oferece um conjunto de garantias que explicam a sua adoção em contextos empresariais, onde a perda de uma mensagem pode ter consequências significativas.

Podem ser gerados erros quando as mensagens não conseguem ser entregues a nenhum subscritor, o que evita perdas silenciosas, situação particularmente perniciosa, por passar despercebida. As mensagens podem ser consumidas de forma que o broker as retenha até existir confirmação de receção por parte do subscritor. É possível publicar mensagens indicando que devem ser serializadas em disco, o que garante a sua sobrevivência a um reinício do broker. O protocolo suporta transações na publicação e confirmação, sob a forma de

operações atómicas. E dispõe de controlo de fluxo, usado, entre outros fins, para evitar que um produtor rápido sobrecarregue um consumidor lento.

O facto de ser uma norma aberta traz vantagens frequentemente subestimadas. Facilita a interoperabilidade entre múltiplas implementações. Evita a dependência de um fornecedor único, tornando mais simples substituir componentes individuais por alternativas. Permite que terceiros escrevam bibliotecas de cliente para outras linguagens, alargando o alcance do protocolo. Permite que ferramentas independentes de análise de tráfego inspecionem e decodifiquem as interações entre clientes e brokers, o que é valioso no diagnóstico e na auditoria de segurança. E promove semelhanças entre as interfaces de programação das diferentes implementações, reduzindo o esforço de migração.

Entre as implementações disponíveis contam-se o Apache Qpid, projeto de código aberto da Apache Foundation que mantém suporte para várias versões do protocolo; o Apache ActiveMQ, também da Apache Foundation; o RabbitMQ, o mais amplamente utilizado e o adotado nesta unidade curricular; os serviços geridos Azure Event Hubs e Azure Service Bus, da Microsoft; o Solace PubSub, broker multiprotocolo disponível em hardware, software e nuvem; o JORAM, implementação Java de código aberto do consórcio OW2; e o StormMQ, serviço comercial.

3 MQTT e Comparação Final

Este capítulo trata o MQTT, o protocolo de mensagens mais usado em soluções de Internet das Coisas, e conclui com a comparação entre os dois protocolos e os critérios de seleção.

3.1 MQTT: Conceito e Qualidade de Serviço

O MQTT (*Message Queue Telemetry Transport*) é descrito como um protocolo de mensagens leve, baseado em publicação-subscrição, para utilização sobre a pilha TCP/IP. Exige a existência de um broker de mensagens e está normalizado como ISO/IEC 20922. Caracteriza-se por um código de dimensão reduzida, o que o torna implementável em dispositivos com pouca memória, e foi concebido para ambientes com largura de banda limitada, precisamente os ambientes restritos típicos da Internet das Coisas. Foi desenvolvido pela IBM em 1999 e disponibilizado sem custos de licenciamento em 2010. É agnóstico quanto aos dados: não impõe qualquer formato à carga útil das mensagens, deixando essa decisão à aplicação.

O aspeto do protocolo com maior impacto prático são os três níveis de qualidade de serviço, escolhidos mensagem a mensagem.

No nível QoS 0, correspondente a “no máximo uma vez”, o broker ou o cliente entrega a mensagem uma única vez, sem qualquer confirmação. É o nível de menor custo e adequa-se a leituras periódicas em que a perda ocasional é irrelevante. No nível QoS 1, “pelo menos uma vez”, a mensagem é entregue com confirmação exigida; garante que a mensagem chega, mas admite entregas duplicadas se a confirmação se perder, o que obriga a aplicação a tolerar duplicados. No nível QoS 2, “exatamente uma vez”, a entrega única é garantida através de um protocolo de quatro passos; é o nível mais fiável e o mais dispendioso em tráfego e latência.

A possibilidade de escolher o nível mensagem a mensagem permite reservar o custo mais elevado para os dados que efetivamente o justificam. Um comando que atua sobre um equipamento (abrir uma válvula, desligar um motor) justifica QoS 2, porque tanto a perda como a duplicação teriam consequências reais. Uma leitura de temperatura enviada a cada minuto raramente justifica mais do que QoS 0.

Uma segunda funcionalidade é a das mensagens retidas. O broker pode reter a última mensagem publicada em cada tópico, e um cliente que subscreva o tópico posteriormente recebe-a de imediato, sem ter de aguardar pela publicação seguinte. Isto é particularmente útil quando o intervalo entre publicações é longo, porque um novo consumidor conhece desde logo o último estado conhecido do sensor, em vez de ficar sem informação durante minutos.

Os tópicos organizam-se hierarquicamente, com níveis separados por barras, e admitem caracteres universais na subscrição. Esta organização permite estruturar o espaço de nomes de forma significativa (por instalação, por divisão, por tipo de grandeza medida) e um consumidor pode assim subscrever todos os sensores de temperatura de um edifício com uma única subscrição.

Quanto à adoção, o protocolo tem presença em contextos muito diversos: o Facebook Messenger recorreu ao MQTT no transporte de mensagens da aplicação móvel; a Amazon Web Services anunciou em 2015 o serviço Amazon IoT baseado em MQTT; a especificação SensorThings API do Open Geospatial Consortium inclui uma extensão MQTT, demonstrada num projeto-piloto do Departamento de Segurança Interna dos Estados Unidos; os serviços de infraestrutura do OpenStack são interligados por um barramento unificado de mensagens MQTT, com o Mosquitto como broker; e a Adafruit disponibiliza um serviço público baseado em MQTT para projetos de IoT.

3.2 Comparação, Critérios de Seleção e Síntese

A comparação entre AMQP e MQTT organiza-se em cinco eixos.

Em complexidade, o AMQP oferece um modelo mais rico, com *exchanges*, filas e transações, enquanto o MQTT se limita a publicação-subscrição sobre tópicos. Em sobrecarga de protocolo, o MQTT foi concebido para a minimizar, sendo por isso mais adequado a ligações de largura de banda reduzida. Em garantias, o AMQP oferece transações atómicas, controlo de fluxo e serialização em disco, ao passo que o MQTT oferece três níveis de qualidade de serviço e

mensagens retidas. Em modelos suportados, o AMQP cobre filas e publicação-subscrição, e o MQTT apenas o segundo. Em vocação, o AMQP orienta-se para interoperabilidade empresarial e integração entre sistemas, enquanto o MQTT se orienta para dispositivos restritos e redes instáveis.

Existem estudos comparativos publicados que avaliam ambos os protocolos em redes instáveis e móveis, considerando eixos como dimensão de mensagem contra sobrecarga de protocolo, consumo energético contra requisitos de recursos, e largura de banda contra latência. A leitura destes trabalhos é recomendada a quem tenha de fundamentar uma escolha em contexto profissional.

Na prática, o critério de seleção pode resumir-se assim: se a solução vive sobretudo do lado dos dispositivos restritos e da rede de sensorização, o MQTT é a escolha natural; se a solução tem de integrar-se com sistemas empresariais existentes e exige garantias transacionais, o AMQP justifica a complexidade adicional. Não é raro que ambos coexistam numa arquitetura, com MQTT do lado dos dispositivos e AMQP na integração empresarial, ligados por uma ponte entre brokers.

Em resumo: o middleware orientado a mensagens resolve um problema de arquitetura, não de transporte, ao desacoplar quem produz dados de quem os consome no espaço, no tempo e na sincronização. É esse desacoplamento que permite acrescentar mecanismos de análise, alarmística ou armazenamento sem alterar os dispositivos que recolhem os dados, exatamente o requisito de escalabilidade formulado no enunciado do mini-projeto. O AMQP é uma norma aberta com modelo rico, vocacionada para integração empresarial; o MQTT é um protocolo deliberadamente minimalista, vocacionado para dispositivos restritos. Ambos assentam em TCP e exigem um broker. A aula seguinte aplica estes conceitos, ligando uma rede Zigbee a um broker MQTT e fechando o percurso do sensor ao middleware.